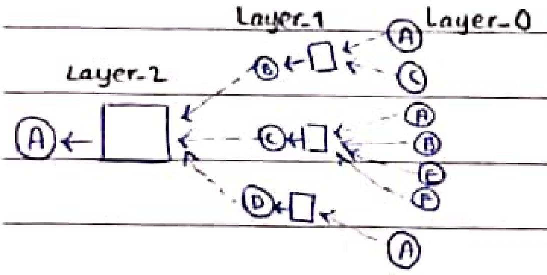
جلسه بیست و سوم - ۱۴۰۰/۹/۲۱

می‌خواهیم دقیق‌تر تعریف کنیم داخل box هایی که جلسه پیش گفتیم چه چیزی است. می‌توانیم در نظر بگیریم یک تابع f است که ورودی‌اش یک multiset از embedding ها است در لایه پایین و خروجی‌اش یک بردار embedding است. معمولا داخل این ها از شبکه‌های عصبی استفاده می‌شود. مثلا می‌تواند پرسپترون، MLP، LSTM و ... باشد.

گفتیم این فرآیند که embedding نود با لایه‌های مختلف با استفاده از embedding های قبلی خودش و همسایه‌هایش می‌سازیم برای هر نود یک Computational Graph حاصل می‌شود. به هر کدام از لایه‌های یک GNN Layer گفته می‌شود. Computational Graph ها بر اساس neighborhood ای که برای هر نود ساخته‌ایم ساخته می‌شود. از آن جایی که نودهای مختلف همسایگی‌های متفاوتی دارند، Computational Graph هایشان هم با هم فرق دارند و isomorphic نیستند اما عملیاتی که داخل box ها صورت می‌گیرد (f) یکسان است.



لایه صفر embedding ها

اگر از نود A شروع کنیم، لایه ماقبل آخر همسایه‌های نود (معمولا خود A) هستند. همسایه‌های مستقیمش (1-hop).

یک لایه عقب‌تر برویم همسایه‌های همسایه‌های A هستند اصطلاحا 2-hops گفته می‌شود که فاصله ۲ دارند از A.

اگر k تا لایه داشته باشیم برای ساختن embedding نود A، نودهایی نقش دارند که فاصله‌شان از A حداکثر k است.

الگوریتم‌های مختلف خیلی وقت‌ها در نحوه‌ی تعریف box ها هست که با هم تفاوت دارند.

همانطور که گفتیم embedding های نودهای مختلف از یک شبکه خطی عبور می‌کنند و به یک message تبدیل می‌شود. بعد این message ها با هم میانگین گرفته می‌شود و به یک شبکه عصبی داده می‌شود یا اینکه خود این میانگین‌گیری توسط شبکه عصبی انجام می‌گیرد.

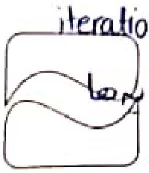
آن چیزی که در اولین شبکه‌ی عصبی گراف بیان شد چنین چیزی بود:

$h_v^0 = X_v$ لایه‌ی صفرم embedding نود v برابر با X_v است. X_v بردار ویژگی‌های نود v است.

$Z_v = h_v^{(L)}$ embedding هایی که بعد از L لایه برای نود v ایجاد می‌شود برابر Z_v است:

Convolution به صورت زیر تعریف می‌شود:

$$\boxed{h_v^{(l+1)} = \sigma\left(W_l \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}\right), \forall l \in \{0,1,\dots,L-1\}}$$



در این رابطه ابتدا همسایه‌های نود v را در نظر می‌گیریم ← N(v). روی همسایه‌های نود v یک iteration انجام می‌دهیم به ازای هر همسایه مثل u، تعبیه‌ی نود u در لایه‌ی قبل را در نظر می‌گیریم و آن را بر اندازه‌ی همسایه‌ها تقسیم می‌کنیم و این‌ها را با هم جمع می‌کنیم. در واقع میانگین‌گیری را انجام می‌دهیم. بعد آن را از یک

تبدیل خطی (W_ℓ) عبور می‌دهید. W_ℓ یک ماتریس وزن است. این ماتریس پارامتر ما است و یاد گرفته می‌شود
اگر برای سادگی فرض کنیم که بردارهای تعبیه در هر لایه m تا درایه دارند ماتریس W_ℓ ابعادش $m \times m$ است.
بعد embedding خود نود v در لایه‌ی قبل یعنی $h_v^{(\ell)}$ را در نظر می‌گیریم و آن را از تبدیل خطی عبور می‌دهیم
B_ℓ یک ماتریس است که مانند W_ℓ باید train شود و پارامتر است ($m \times m$ است)
علت این که B_ℓ و W_ℓ دارای index هستند این است که ماتریس‌های وزن در لایه‌های مختلف متفاوت هستند.

ما در یک لایه برای همه‌ی نودها ماتریس‌های وزن یکسانی را استفاده می‌کنیم ولی در لایه‌های مختلف ماتریس‌های وزن متفاوت اند.

حاصل $B_\ell h_v^{(\ell)}$ یک بردار است، همچنین حاصل $W_\ell \sum \frac{h_u^{(\ell)}}{|N(v)|}$ یک بردار است.
بعد روی این دو بردار یک aggregation انجام می‌دهیم مثلا جمع
در نهایت بردار (در فضای m بعدی) داریم و آن را به تابع σ می‌دهیم که نا‌خطی‌سازی انجام می‌دهد.

نکته‌ی ۱: اگر ما L لایه داشته باشیم و در هر لایه یک بردار embedding در فضای m بعدی تولید کنیم تعداد پارامترها برابر با $2 \times L \times m \times m$ است. تعداد لایه‌ها معمولا ثابت است، m هم ثابت می‌گیریم چون می‌خواهیم بردارها در فضایی با ابعاد کم باشند. در روش‌های shallow تعداد پارامترهایی که باید یاد گرفته شوند $O(|V| \times m)$ است که m را ثابت می‌گیریم پس $O(|V|)$ است. در حالی که در روش‌های GNN از مرتبه $O(1)$ است.

نکته‌ی ۲: قبلا گفته بودیم که اول از تبدیل خطی عبور می‌دهیم بعد aggregation می‌گیریم اما در فرمول بر عکس آورده شده علتش این است که ترتیب این‌ها فرقی ندارد. تبدیل‌های خطی نسبت به linearity بسته هستند.

نکته ۳: به سری embedding‌های میانی که در این روند تولید می‌شوند که به آن‌ها hidden embedding می‌گویند. که توسط hidden layers تولید می‌شوند
برای یافتن وزن‌های B_ℓ و W_ℓ یک تابع هدف تعریف می‌کنیم و با روش SGD آن را بهینه می‌کنیم.

ما objective function مان را بسته به setting‌های مختلف به شیوه‌های متفاوتی تعریف کنیم
setting ممکنه supervised باشه یا unsupervised

وقتی setting ما supervised است اصلا می‌توانیم این setting را کنار بگذاریم و embedding‌ها را به صورت کلی تعریف کنیم. یا یک loss function مخصوص خود embedding‌ها جلو ببریم. بعد این embedding‌های به دست آمده را به یک classifier بدهیم. علتش این است که نمی‌خواهیم embedding‌ها را برای یک تسک supervised خاص یاد بگیریم بلکه تسک‌های دیگری هم انجام دهند. پس یک setting داریم که دیتای برچسب‌دار داریم و می‌خواهیم با استفاده از آن تابع loss را تعریف کنیم. یک setting دیگر هم هست که دیتای برچسب‌دار نداریم یا اینکه داریم ولی نمی‌خواهیم از آن استفاده کنیم. (دومی: unsupervised)
در حالت اول برای هر example یک لیبل واقعی و لیبل تخمین زده شده را داریم: y و $f(z_v)$
تابع loss ما این جفت را ورودی می‌گیرد

$$\min_{\Theta} L(y, f(z_v))$$

در حالت unsupervised دیتای که لیبل داشته باشند نداریم در این حالت structure خود گراف را

به عنوان supervision استفاده کنیم. مشابه همان کاری که در روش های shallow انجام می دادیم.
یعنی می گوییم شباهت دو نود در گراف باید نزدیک باشد به شباهت بردارهای embedding آن ها.
یک Loss function ای که استفاده می شود:

$$L = \sum_{z_u, z_v} CE(y_{u,v} , DEC(z_u, z_v))$$ (Unsupervised Training)

روی جفت نودها iteration می زنیم. به ازای هر جفت نود یک تابع cross entropy حساب می کنیم بعد این CE ها را برای همه ی جفت نودها با هم جمع می کنیم.

$y_{u,v}$ ← شباهت نود u و v در معیارهای گرافی (مثلا hitting time, personalized pagerank, ...)

$DEC(z_u, z_v)$ ← میزان شباهت میان بردار تعبیه ی z_u و z_v (مثلا ضرب داخلی)

CE مشخص می کند این دو مقدار چقدر به هم شبیه اند. cross entropy شباهت دو تا بردار را اندازه می گیرد. در این جا CE در واقع شباهت بین این دو تا شباهت را به ما می دهد.

CE منفی شباهت را می دهد که می خواهیم آن را min کنیم

Supervised Training

در این حالت برامون مهم نیست که embedding های ما similarity preserving باشند بلکه می خواهیم تسک classification را خوب انجام بدهند

مقاله ی ICLR 2017 از این Loss function استفاده کرده: (یک نسخه ای از CE است)

$$L = \sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1-y_v) \log(1-\sigma(z_v^T \theta))$$

فرض کنید دو تا کلاس (۰ و ۱) داریم در این تابع هدف. ما نه تنها برای هر دیتا پوینت یک embedding می خواهیم یاد بگیریم بلکه برای هر کدام از کلاس ها هم یک بردار embedding می خواهیم یاد بگیریم.

کلاس ۱ یک بردار embedding دارد ← θ کلاس ۰ یک بردار embedding دارد ← $1-\theta$

پس در کنار ماتریس های وزنی که قبلا داشتیم این دو تا پارامتر هم اضافه می شود

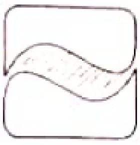
v ← یک Training example (با برچسب معلوم) y_v ← برچسب واقعی example

اگر $y_v = 1$ ← شباهت بین بردار تعبیه ای که برای v یاد گرفته می شود و بردار تعبیه ای که برای کلاس ۱ داریم باید max شود. در رابطه بالا جمله دوم حذف می شه ← ۰ فقط کمیت اول می مونه ($y_v \log(\sigma(z_v^T\theta))$)

$z_v^T \theta$ ← شباهت بین بردار تعبیه ی نود v و بردار تعبیه ی کلاس ۱

یعنی $y_v = 0$ ← شباهت بین بردار امبدینگ نود v و بردار امبدینگ کلاس ۰ باید max بشه

← جمله اول صفر، جمله دوم ← منفی شباهت بردار تعبیه v و بردار تعبیه کلاس ۱ =



معمولا از [illegible] که پارامترها هستند [illegible] می گیریم به nonlinearity هم induce کردن

بصورت تجربی که اگه نداریم کمتره

Scanned with CamScanner

مزیت های GNN نسبت به روش های shallow

امتیاز اول: تعداد پارامترهاشون کم‌تره

امتیاز دوم: خاصیت inductivity دارند:

گراف شما ممکن است خیلی بزرگ باشد. یک امتیاز این روش این است که وزن‌های W_l و B_l مستقل از نودهای شما هستند به عبارتی وزن‌های شما بین نودهای مختلف مشترک (shared) هستند. چنانچه گراف خیلی بزرگ باشد گرافتان که می‌خواهید نمونه‌برداری تک تک نودها iteration باید انجام بدهید که نمی‌شه. میایید یه قسمتی از گراف را انتخاب می‌کنید، فرآیند آموزش رو فقط روی این قسمت انجام می‌دهید (٤)

① و وزن‌ها را فقط برای این قسمت یاد می‌گیرید. به سادگی وزن‌ها رو می‌گیرید بر حسب این‌ها که برای کل گراف به کار می‌برید.

(۳ امتیاز inductivity)

② حتی می‌توانید از یک گراف به یک گراف دیگر بروید البته در مواردی خوب جواب می‌دهد که گراف‌های شما از یک category و خانواده باشند. یه گراف خاص رو انتخاب می‌کنید، training را روی آن انجام دهید وزن‌ها را روی آن یاد می‌گیرید بعد این وزن‌ها را می‌توانید برای هر گراف دیگه‌ای از اون category استفاده کنید مثلا protein interaction

③ یک امتیاز دیگر Inductivity این است: شبکه‌های پیچیده دنیای واقعی معمولا dynamic هستند یعنی ساختارشان با یه سری عملیات آپدیت تغییر می‌کند (اضافه کردن نود، حذف نود، اضافه کردن یال، حذف یال، تغییر وزن یال) از این خاصیت Inductivity می‌توانید استفاده کنید برای نود جدید embedding را محاسبه کنید بدون نیاز به اینکه training را از سر انجام دهید. می‌توانید مدلی تعریف کنید که به صورت incremental تغییر پارامترهای جدید را محاسبه کند.

GraphSAGE

در NIPS ، 2017 پیشنهاد داده شد. نویسنده‌های GCN عمل aggregation را این گونه پیشنهاد دادند که میانگین بردارهای همسایه را با بردار خود نود از لایه قبلی جمع کنند.

در Graphsage گفتند چرا خودمان را به آن تابع Agg خاص محدود کنیم؟

رابطه‌ی AGG را GraphSAGE پیشنهاد داد:

$$h_v^{(l+1)} = \sigma\left(\left[W_l \cdot AGG\left(\{h_u^{(l)}, \forall u \in N(v)\}\right), B_l h_v^{(l)}\right]\right)$$

یک تابع AGG عمومی استفاده می‌شه برای اینکه همسایه‌های نود رو تبدیل کنیم به یک بردار. بعد نتیجه را از تبدیل خطی عبور می‌دهید. همچنین تعبیه‌ی خود نود را با یک تبدیل خطی عبور کرده. و یک Agg function بین تعبیه‌ی خود نود و تعبیه‌ی حاصل از Agg function عمومی (همسایه‌ها) تعریف می‌کند.

یک پیشنهاد دیگر این مقاله: در انتهای هر لایه بعد از اینکه تعبیه نود را حساب کردیم یک نرمال‌سازی هم داشته باشیم. بردار تعبیه را نرمال کنیم با نرم ۲، یعنی نرم ۲ اش یک باشه.

$$h_v^k \leftarrow \frac{h_v^k}{\|h_v^k\|_2}$$

نرم 2 : $\|u\|_2 = \sqrt{\sum_i u_i^2}$

نرمال‌سازی باعث می‌شه embedding ها همشون نقاطی روی یک کره در فضای d بعدی باشن

نویسنده‌های این مقاله ۳ تا تابع AGG پیشنهاد دادند:

① Mean → همان چیزی که در GCN پیشنهاد شد

$$AGG = \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|}$$

② Pool استفاده از یک MLP

بردارهای تعبیه همسایه‌ها را از یک MLP عبور می‌دهید مثلا ۳ بردار ورودی می‌دهید ۳ بردار خروجی دارید. بعد یک AGG function element wise اعمال می‌کنید مثلا max گرفتن خانه‌های متناظر، Mean ، Sum و ...

$$AGG = \gamma\left(\{MLP(h_u^{(l)}), \forall u \in N(v)\}\right)$$

حالا این AGG function مثلا MLP ای که اینجا استفاده شده پارامترهایی دارد، مثلا ماتریس وزن دارد، ماتریس‌های وزنش هم‌زمان با پارامترهای اصلی مدل train می‌شوند. (هم‌زمان با تبدیل خطی‌هایی که در لایه‌های مختلف داریم یاد گرفته می‌شوند)

③ LSTM ← جاهایی استفاده می‌شه که ترتیب وجود دارد بین ورودی‌ها. مثلا یعنی ترتیب (زمانی یا مکانی)

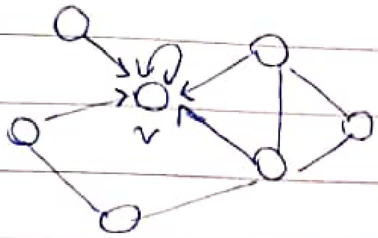
یک ترتیب تصادفی از بردارهای ورودی ایجاد می‌کنیم. بعد این ترتیب را به LSTM می‌دهید

خروجی LSTM در نهایت یک تک بردار است. این بردار یاد می‌گیرد که از هر کدام از بردارهای ورودی ما چه مواردی نگه داشته بشن و چه مواردی فراموش بشن و بردار نهایی را می‌سازد.

$$AGG = LSTM([h_u^{(l)}, \forall u \in \pi(N(v))])$$

GraphSage به ایراد مهمی که در پیاده سازی GCN بود را برطرف کرد. اینکه ماتریس مجاورت را همیشه می‌ساخت.

جلسه بیست و چهارم - ۱۴۰۰/۹/۲۳



می‌توانیم شکل روبه‌رو را یک جلسه‌ی مشورتی در نظر بگیریم
در ابتدا نود وسط یک شورای مشورتی برای خودش می‌سازد
تا embedding اش را تشکیل بدهد.

اعضای این شورا نظراتشان را در قالب message در می‌آورند و به نود v که این جلسه را ساخته می‌گویند.

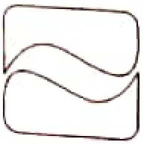
نود v نظرات خودش و اعضا را دریافت می‌کند. هر کدام از این‌ها وزنی دارند. این‌ها را با هم AGG می‌کند و نظر نهایی که همان تعبیه‌ی نود v در لایه‌ی l هست را می‌سازد.

می‌خواهیم یک فریم‌ورک دید کلی در مورد شبکه‌های عصبی گرافی ارائه بدهیم.
در همین جهت می‌خواهیم ببینیم که کامپوننت‌های مختلف شبکه عصبی گرافی چه مواردی هستند
در مورد مکانیزم attention نیز در شبکه‌های عصبی گرافی صحبت می‌کنیم

اگر یک فریم‌ورک برای GNN ها در نظر بگیریم به طور کلی چند تا کامپوننت اصلی دارد:

① Message ← یک embedding از سطح k-1 را می‌گیرد (از یک نود) و آن را با تبدیل خطی تبدیل به message می‌کند تا embedding سطح k را بسازد
در حقیقت یک ماتریس وزن trainable داریم که روی بردار تعبیه‌ی سطح پایین‌تر اعمال می‌کنیم
تبدیل message معمولا توسط تبدیل خطی انجام می‌گیرد. راه‌های دیگری هم هست

② Aggregation ← عملگر ساده و بدون پارامتر باشد مثل mean، max، min یا عملگر از نوع شبکه عصبی استفاده شود مثل MLP یا LSTM که نیاز به پارامتر trainable داشته باشد.



③ Layer Connectivity ← تا الان هر لایه ما یک message یک aggregation داشت.
حالت ساده اینکه هر لایه به لایه بعدی اش متصل باشد اما یک سری مشکلات به وجود می‌آورد

← ادامه در دفتر

شنبه
Saturday
1 October 2011
۳ ذی القعده ۱۴۳۲

④ Graph Augmentation (افزونه سازی گرافی)

تا این جا فرض ما این بوده که از روی گراف خام میاید شبکه عصبی گرافی روی سازید مثلا از روی گراف شبکه اجتماعی ولی این کار یا امکان‌پذیر نیست یا down grade میکنه GNN را.

شما باید اول یه سری عملیات روی گراف ورودی انجام دهید بعد از روی آن GNN بسازید برای هر نود یک Computational Graph بسازید و از روی آن تعبیه‌ها را.

به این عملیات میگن Graph Augmentation به طور کلی. ۲ دسته‌اند:

① Graph Feature Augmentation ← فیچرهای نودها را دستکاری می‌کنند و باهاشون کار می‌کنند

② Graph structure ″ ← ساختار شبکه‌های گراف را دستکاری می‌کنه

روز همبستگی و همدردی با کودکان و نوجوانان فلسطینی - روز جهانی سالمندان

۱۰
یکشنبه
Sunday
2 October 2011
۴ ذی القعده ۱۴۳۲

⑤ objective function

unsupervised / supervised

Node / Edge / Graph

تعبیه‌ها را میتونه در سطح نود، یال یا گراف استفاده کنید. آن چه تا به حال دیدیم در سطح نود بوده میتونه توسعه بدهید به یال‌ها یا کل گراف. اینکه توسعه چگونه صورت بگیرد و تابع هدف چه چیزی باشد؟

بیرونه GNN Layer دو تا کامپوننت اصلی‌اش Message و agg است

ورودی GNN Layer ← یک مالتی ست از بردارهای embedding در سطح پایین تر

خروجی GNN Layer ← یک Single embedding

در حالت کلی یک تابع message می توانیم داشته باشیم. در مثال این تابع توسط یک ماتریس وزن trainable تعریف می شود.

تابع message یک بردار تعبیه در سطح پایین تر را می گیرد و یک message در سطح جدید به ما می دهد

$$m_u^{(l)} = MSG^{(l)}(h_u^{(l-1)})$$

ممکنه message و ورودی هر دو در یک فضا باشند یا در فضاهای مختلف

ورودی AGG مجموعه ای از message ها که در لایه جدید تولید شده اند (از روی تعبیه های لایه قبل)

گفتیم فرآیند معمولا GNN layer را می توان به صورت یک جعبه ی مشورتی در نظر گرفت

مشکلی که هست اینه که نظر خود شخص اهمیت ویژه ای دارد به همین دلیل یک ماتریس وزن جداگانه برای آن در نظر می گیریم ← B همسایه ها ← W

معمولا B ماتریس بزرگتری است و وقتی train می شود نرم F بزرگتری دارد، که نشان دهنده اهمیت بیشتر خود نود است

در اینجا اعضای مشورتی یک جمع بندی به دست می آید بعد این جمع بندی را با نظر خود نود ($m_v^{(l)}$) مقایسه و جمع بندی نهایی می کنیم

$$h_v^{(l)} = CONCAT(AGG(\{m_u^{(l)}, u \in N(v)\}), m_v^{(l)})$$

(در واقع تابع AGG داریم) یعنی که روی همسایه ها اعمال می گیره

ولی هم که این ها concat گذاشته

در نهایت خیلی وقت ها یک nonlinearity هم داریم که یعنی تابع غیرخطی ساز را روی تک تک خروجی خانه های اعمال می کنه. (σ) ← مثلا Sigmoid ، ReLU

البته جای سوال است که غیرخطی ساز در GNN ها مورد نیاز هست یا نه؟

در شبکه های عصبی معمولی خیلی مهمه چون اگه نباشه می شه لایه های خطی را collapse کرد اما در GNN ها اینطور نیست حتی اگه نباشه collapse نمی شه به خاطر ساختار Computational Graph ای که داره و AGG ها که انجام می دهیم.

یک الگوریتم به نام SGN هست که استفاده نمی کنه از nonlinearity ← زمان اجرای training خیلی پایین است اما دقت هم چنان تا حد خوبی بالا است

از طرفی یک سری مطالعات نشون داده برای expressive power شبکه های عصبی گرافی به nonlinearity خیلی وابسته است.

چگونه می‌توانیم GCN را در قالب message + AGG بیان کنیم؟

رابطه Convolution در GCN این بود:

$$h_v^{(l)} = \sigma\left(W^{(l)} \sum_{u \in N(v)} \frac{h_u^{(l-1)}}{|N(v)|}\right)$$

N(v) در GCN خود نود v را هم در بر دارد

در فرمول اول AGG داریم بعد message

ولی وقتی message ها تبدیل خطی به دست میاد و AGG مثلا sum باشه تبدیل خطی می‌تونه از داخل sum در بشه

$$h_v^{(l)} = \sigma\Big(\underbrace{\sum_{u \in N(v)} \underbrace{W^{(l)} \frac{h_u^{(l-1)}}{|N(v)|}}_{\text{Message}}}_{\text{Aggregation}}\Big)$$

message $\leftarrow m_u^{(l)} = \frac{1}{|N(v)|} W^{(l)} h_u^{(l-1)}$

$$h_v^{(l)} = \sigma\big(\underbrace{\text{sum}}_{AGG}(\{m_u^{(l)}, u \in N(v)\})\big)$$

چگونه GraphSAGE را در این قالب بیان کنیم؟

$$h_v^{(l)} = \sigma\big(W^{(l)} \cdot \text{CONCAT}(h_v^{(l-1)}, \underbrace{\text{AGG}(\{h_u^{(l-1)}, \forall u \in N(v)\})}\,)\big)$$

message داخل این تابع AGG انجام می‌گیره، هر کدوم از همسایه‌ها را تبدیل به یک message می‌کنیم مثلا اگه از MLP استفاده کنیم هر تعبیه را از یک mlp عبور می‌دهیم بعد یک sum یا Avg روی این‌ها را می‌زنند.

این‌جا هم تبدیل message داخل تابع AGG انجام می‌گیره وقتی که هر تعبیه‌ای را از MLP عبور می‌کنه

تابع AGG هم دو مرحله‌ای است: ابتدا همسایه‌ها را باهم، بعد خروجی اش را با خود نود v ← concat

① $h_{N(v)}^{(l)} \leftarrow \text{AGG}(\{h_u^{(l-1)}, \forall u \in N(v)\})$

② $h_v^{(l)} \leftarrow \sigma(W^{(l)} \cdot \text{CONCAT}(h_v^{(l-1)}, h_{N(v)}^{(l)}))$

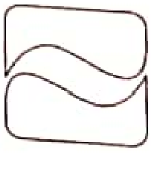
(GAT) Graph Attention Networks

ممکنه نظرات افرادی که در شورای مشورتی هستند اهمیت یکسانی برای ما نداشته باشد.

تا این جا برای تمام همسایه‌ها وزن یکسانی می‌گرفتیم (ماتریس W)

در GCN برای خود نود و همسایه‌ها وزن یکسان می‌دهیم.

فرض کنید برای هر همسایه u وزن α_{vu} بدهیم و حتی ممکنه این وزن‌ها در لایه‌های مختلف با هم فرق کنند به همین دلیل $\alpha_{vu}^{(l)}$ در نظر بگیریم



در GraphSage وزن‌های متفاوتی برای همسایه‌ها به صورت explicit تعریف می‌شود

α_{vu} ← اهمیتی که نود u برای نود v دارد ← وقتی ساختار شبکه‌ای برای اونایی که α بزرگتری دارند در نظر گرفته می‌شود.

در مکانیزم توجه این وزن‌های α به صورت implicit در نظر گرفته می‌شوند و trainable هستند

تابع a را ببینیم و ورودی‌هایش: message نود u در لایه l ام
〃 〃 〃 〃 v 〃 〃

خروجی یک عدد که وزن بین u و v است

$$e_{vu} = a(\underbrace{W^{(l)} h_u^{(l-1)}}, \underbrace{W^{(l)} h_v^{(l-1)}})$$

وزن u برای v را نشان می‌دهد

بعد این وزن e_{vu} نرمال می‌شود و α_{vu} به دست می‌آید با استفاده از softmax

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

بعد این α استفاده می‌شود در فرآیند aggregation هر همسایه متناسب با وزنش در agg شرکت می‌کند:

$$h_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} W^{(l)} h_u^{(l-1)}\right)$$

تابع a چگونه تعریف شود؟ مکانیزم توجه چگونه باشد؟

یک پیشنهاد: message ها را concat کنید (یعنی embedding ها را) و در نهایت یک تبدیل خطی روی این بردار concat شده اعمال کنید.

تبدیل خطی روی یک بردار با ابعاد 2d اعمال می‌شود و حاصل باید یک اسکالر باشد پس ماتریس وزن باید 1×2d باشد

$$e_{AB} = a(W^{(l)} h_A^{(l-1)}, W^{(l)} h_B^{(l-1)}) = \text{Linear}(\text{Concat}(W^{(l)} h_A^{(l-1)}, W^{(l)} h_B^{(l-1)}))$$

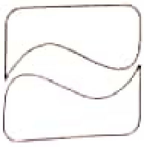
این ماتریس وزن 1×2d هم جزو پارامترهاست و train می‌شود. این ماتریس‌ها بین نودهای مختلف یکسان‌اند ولی برای لایه‌های مختلف ممکنه متفاوت تعریف بشن

اگر از چندتا مکانیزم توجه استفاده کنیم و در آخر نتایج را agg کنیم نتایج خیلی کمتر می‌شود.
فرض کنید ۳ تا مکانیزم توجه استفاده می‌کنید یعنی ۳ تا تابع مختلف برای محاسبه a استفاده می‌کنید
α^1 و α^2 و α^3 که هر کدام embedding متفاوتی می‌دهد و بعد این‌ها را agg می‌کنید

$$h_v^{(l)}[1] = \sigma\left(\sum_u \alpha_{uv}^1 W^{(l)} h_u^{(l-1)}\right)$$

$$h_v^{(l)}[2] , h_v^{(l)}[3] \qquad h_v^{(l)} = AGG(h_v^{(l)}[1], h_v^{l}[2], h_v^{l}[3])$$

این فرآیند چند امتیاز دارد: ① از نظر محاسباتی کمینه است
چون فقط وزن همسایگی نودها باید حساب کرد و خیلی localized است، وزن‌ها وزن‌های مختلف برای نودهای مختلف را می‌توان به صورت موازی محاسبه کرد. در نهایت از SGD استفاده می‌کنید، مشتق‌ها را که بگیرید از کل عبور می‌کند یعنی برای نودهای مختلف میشه parallel انجام داد



② از نظر فضای ذخیره‌سازی کمینه است چون برای ذخیره‌ی وزن‌ها اگر گراف ما متصل باشد $O(|E|)$ نیاز دارید. هر نود و همسایه‌های هر نود را ذخیره می‌کنید $O(2|E|)$

تعداد پارامترها هم ثابت است. در ستون هر جفت نود وزن داریم این وزن را با تابعی یاد می‌گیریم که دارای پارامترهای shared است

③ inductivity ← ۲ نوع الگوریتم داریم: ۱) میزان شباهت بین دو تا نود را به عنوان پارامتر می‌گیره و یاد می‌گیرند. مقدار پارامترها زیاده، نتایج خوبه ولی transductive است یعنی با ورود یال جدید بین دو نود شما در فاز تست نمی‌دانید وزن آن را چگونه پیدا کنید

۲) میزان شباهت بین دو نود بر حسب یه سری پارامترهای مشترک (shared) یاد گرفته می‌شود اگر یال جدیدی هم اضافه شد میزان شباهت راحت محاسبه می‌شه. مکانیزم attention اینگونه است. ← setting شما قابل تعمیم است به حالت‌هایی که یال‌های جدید یا گراف جدید دارید

فضای طراحی شبکه‌های عصبی گرافی

تا الان نگاه basic داشتیم به GNNها، در عمل خیلی کامپوننت‌های زیادی دیگه‌ای هم در لایه GNNها داشته باشیم که تاثیر قابل توجهی در عمل دارند

۱) مثلاً Batch Norm ← در SGD میایم یه Batch از دیتا رو برمی‌داریم و کل دیتا را برای این‌ها سعی می‌کنیم گستره دیتایی که در این Batch هستن نرمال بشوند. وزن‌های یاد گرفته شده را stable می‌کند و نتایج بهتری می‌دهد.

۲) Dropout ← برای جلوگیری از overfitting

به طور کلی یک تک لایه از GNN ممکنه از این موارد ساخته شده باشه:

→ Linear → Batch Norm → Dropout → Activation → Attention → Aggregation →

جلسه بیست و پنجم ۱۴۰۰/۹/۲۸

Batch Normalization

برای این انجام می‌شود که اگر یک فیچری یک مقدار خیلی زیاد داشت به یکباره کل فرآیند learning و ماتریس‌های وزن ما را تحت تاثیر قرار ندهد و کل وزن‌ها و پارامترها مایل نشوند به سمت آن مقدار خیلی زیاد.

در حقیقت میایم مقادیر را نرمالایز می‌کنیم. فرض کنید یک گراف به ما داده‌اند که مقادیر اولیه‌اش برای نودها فیچرش دارد. این مقادیر را با Batch Norm میایم نرمال می‌کنیم.

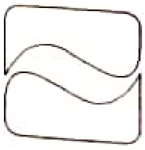
کار خوب اینه که نرمال کردن را روی کل گراف انجام بدهیم اما به دلیل حجم بالا میایم یه Batch رو برمی‌داریم

۱) scale دیتا رو عوض می‌کنیم به طوری که میانگین دیتا صفر بشه ← zero_mean می‌کنیم در هر dimension

۲) واریانس دیتاها تو dimension را ۱ یا نزدیک به ۱ می‌کنیم

ورودی ← N تا node embedding میان که batch را می‌سازند

با ماتریس $X \in R^{N \times D}$ به این batch اشاره می‌کنیم



به ازای هر دیتاپوینت در batch ما یک ردیف وجود دارد، D تعداد dimension هاست

دو هدف داریم:

① در هر dimension میانگین دیتا را صفر می‌کنیم

② واریانس دیتا در آن راستا (dimension) را 1 می‌کنیم

← trainable هستند

دو تا پارامتر داریم در فضای d بعدی که باید آموزش داده شوند: $\gamma, \beta \in \mathbb{R}^D$

فرض ← یک ماتریس از دیتای نرمال شده ← در فضای N×D $Y \in \mathbb{R}^{N\times D}$

Normalization به چه صورت انجام می‌گیرد؟

در گام اول میانگین در هر dimension را حساب می‌کنیم:

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

بعد در هر dimension ژام واریانس را حساب می‌کنیم:

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j}-\mu_j)^2$$

در گام سوم هر دیتای $X_{i,j}$ را برابر با این مقدار قرار می‌دهیم:

$$\hat{x}_{i,j} = \frac{x_{i,j}-\mu_j}{\sqrt{\sigma_j^2+\epsilon}}$$

ϵ به دلیل کاربردهای عملی اضافه شده

اگر واریانس خیلی کوچیک باشه، مثلاً اگه همه‌ی دیتاها در dimension ژام یک مقدار داشته باشند یا خیلی نزدیک به هم باشند، عملیات نرمالیزه کردن عمل می‌کند بدون ϵ

در این کار دیتاهای هر dimension میانگین صفر و واریانس نزدیک به 1 پیدا می‌کنند

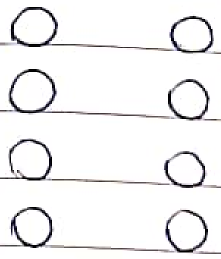
بعد دیتا را Rescale می‌کنیم. برای هر dimension یک γ_j و یک β_j داریم که یاد گرفته می‌شوند (وزن اند).

$$\boxed{Y_{i,j} = \gamma_j X_{i,j} + \beta_j}$$

Dropout for GNNs ← یک ماژول دیگری که استفاده می‌شود در GNN‌ها

هدف: جلوگیری از overfitting، یعنی وزن‌های شبکه به گونه‌ای یاد گرفته شوند که روی داده‌ی train خوب جواب بدهند ولی روی داده‌ی تست عملکرد بدی داشته باشند. برای ^مدل x داده‌ی آموزشی خیلی فیت شده باشد و برای داده‌ی جدید مورد استفاده نباشد.

در شبکه‌های عصبی معمولی در هر لایه تعدادی سلول داریم. فضای d بعدی باید تعداد سلول‌ها d باشد.
تعدادی از این سلول‌ها را به صورت تصادفی غیرفعال می‌کنید (برداری که توزیعی شود را غیرفعال می‌کنید)
فرض کنید برای هر نود u در لایه‌ی l-1 یک embedding یافته‌اید. حالا می‌خواهید به لایه‌ی l بروید.
در گام اول باید تعبیه‌ی نود u را به message تبدیل کنید با استفاده از ماتریس وزن $W^{(l)}$.



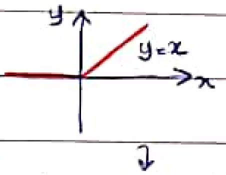
شاید بعضی از خانه‌های تعبیه در لایه‌ی l-1 را به صورت تصادفی بخواهید صفر می‌کنید. و همینطور در لایه‌ی جدید هم بعضی از خانه‌ها را می‌توانید صفر قرار دهید
(روش‌های مختلفی می‌توان استفاده کرد) می‌توان روی لایه l-1 یا l یا هر دو انجام داد
یا روی ماتریس وزن W بعضی خانه‌ها را صفر کنید)

Activation (Non-linearity)

گفتیم خیلی اوقات خروجی ما تابع خطی از ورودی نیست به همین دلیل از Non Linearity استفاده می‌کنیم
این ایده از شبکه‌های عصبی معمولی گرفته شده اما عملکرد این نقش هم که آن جا دارد را این جا داشته باشد
(گفتیم ۲ دیدگاه هست)

ReLU

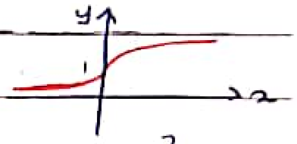
$\mathrm{ReLU}(x_i) = \max(x_i, 0)$



مقادیر هیچ محدودیتی ندارد

Sigmoid

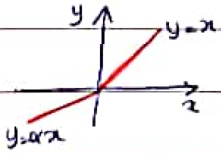
$$\sigma(x_i) = \frac{1}{1+e^{-x_i}}$$



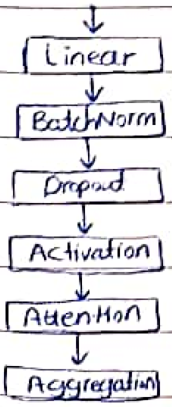
مقادیر تعبیه را بین ۰ و ۱ می‌برد. فقط وقتی می‌توانید از آن استفاده کنید که بخواهید مقادیر بردارهای تعبیه شما بین ۰ و ۱ باشد

parametric ReLU

$\mathrm{PReLU}(x_i) = \max(x_i, 0) + a_i \min(x_i, 0)$



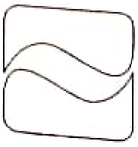
شیب خط در سمت‌های منفی توسط یک پارامتر ورودی α تعیین می‌شود. معمولا این پارامتر هم باید یادگرفته بشه



به طور خلاصه در یک لایه از GNN یه سری کامپوننت‌های استاندارد داریم مثل Linearity که توسط Matrix Transformation، AGG انجام می‌گیره
یه سری ماژول‌های دیگه هم هست اضافه بشه مثل Attention، Dropout، nonLinearity، Batch Norm
(فضای طراحی شبکه‌های عصبی گرافی)

stacking GNN Layers

چگونه لایه‌های مختلف GNN را به هم متصل کنیم؟



یک راه این است که لایه‌ها را پشت سر هم بگذاریم یعنی خروجی لایه پایین‌تر به ورودی لایه بالاتر داده شود
اما مشکل یک سری مسائل پیش می‌آید. خیلی وقت‌ها برای اینکه خروجی خوبی داشته باشیم لازمه ما

ما چند لایه GNN داشته باشیم اما مشکلی به نام over-smoothing به وجود می‌آید یعنی embedding های
همه نودها یکسان یا خیلی شبیه به هم می‌شود. حتی برای نودهایی که خیلی با هم متفاوتند. ما می‌خواهیم تعبیه‌ها
similarity preserving باشند یعنی نودهای متفاوت به embedding های متفاوتی map شوند
چرا این مسئله اتفاق می‌افتد؟ ← اینکه تعداد لایه‌ها وقتی از یک حدی بیشتر می‌شود حتی نودهای متفاوت هم
embedding های یکسان یا خیلی شبیه به هم پیدا می‌کنند؟

Receptive field : مجموعه‌ی تمام نودهایی که روی ساخت embedding یک نود اثر می‌گذارند
اگر یک GNN با k لایه داشته باشیم، کدام نودها در Receptive field یک نود هستند؟
همه نودهایی که حداکثر k تا فاصله دارند از نود مورد نظر.

وقتی تعداد لایه‌ها را کمی افزایش بدهیم تقریباً خیلی زیادی نود داخل receptive field نودها قرار
می‌گیرند. این امر به دلیل پدیده دنیای کوچک (small world) است. نودهای دیگر از نود
مورد نظرها فاصله‌شان خیلی کم است. (۶ درجه جدایی) ← یعنی انسان‌های پیچیده
همچنین با افزایش تعداد لایه‌ها اندازه فصل مشترک دو تا نود هم خیلی سریع افزایش پیدا می‌کند (یعنی با سرعت)
پس ما می‌توانیم پدیده over smoothing را با استفاده از افزایش شدید اندازه‌ی receptive field
توضیح بدهیم.

می‌توانیم این فرض را داشته باشیم که embedding یک نود وابستگی زیادی دارد به receptive field آن نود.
همانطور که دیدیم در این receptive field های مشترک در لایه‌های مختلف خیلی با سرعت بیشتر این امر به وجود می‌آید
ولی در عمل تأثیر چندانی ندارد و به طور کلی receptive field مشترک مهم‌تر است (در هر لایه‌ای که باشد)

در نتیجه ما وقتی تعداد لایه‌ها را زیاد می‌کنیم، receptive field برای دو تا نود دلخواه
اشتراکشان خیلی زیاد می‌شود.
پس embedding هایی که از این دو تا receptive field شبیه به هم تولید می‌شود
خیلی به هم دیگه شبیه می‌شوند و پدیده over smoothing به وجود می‌آید و نودهای مختلف
دارای embedding های یکسان می‌شوند.

چطور می‌توانیم از این پدیده جلوگیری کنیم؟ ← باید تعداد لایه‌ها را زیاد نکنیم یا اگر زیاد می‌کنیم باید دقت کنیم.
① در استفاده از شبکه‌های عصبی گرافی عمیق احتیاط کنید. در بسیاری شبکه‌های عصبی عمیق کردن و
افزایش تعداد لایه‌ها معمولاً مفید است اما در شبکه عصبی گرافی اینگونه نیست.
قانون ← تعداد لایه‌ها باید خیلی کوچکتر از قطر گراف باشد

تعداد لایه ها
↑
قطر گراف کمینه shortest path Length یا Avg path length باشه

چرا باید این کوته باشد؟ چون diameter، فاصله متوسط نودهاست، وقتی k به قطر می رسه یعنی بعد expected همه نودها میان داخل receptive field نود شما قرار می گیرند پس فضای مشترکشان خیلی زیاد می شو

سوال: وقتی قطر گراف کوچک است تعداد لایه ها کم می شود باید چکار بکنیم؟ ۲ تا راه حل:

① هر لایه را تقویت کنید. expressiveness هر لایه را زیاد کنید.

یک کاری که می توانید انجام دهید این است که هر لایه را پیچیده تر کنید. مثلا حتما از مکانیسم Attention استفاده کنید. مکانیزم AGG را پیچیده تر کنید مثلا از MLP استفاده کنید یا از یک شبکه عصبی پیچیده با تعداد لایه زیاد استفاده کنید. (یا LSTM مثلا با ۵ یا ۶ لایه)

این ایده یه سری مزایا داره ← میتونه expressiveness را بالا ببرد.

اما این کار یک سری معایب هم داره: مدل شما را پیچیده می کند. explainability و شفافیت مدل شما را پایین می آورد.

NON_GNN

② یک سری لایه اضافه کنید به خصوص در ابتدا و انتهای شبکه که این لایه ها GNN نیستند.

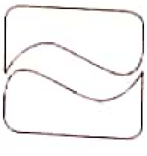
مثلا ابتدای کار فرض دارید که هر نود شما content (محتوا) دارد. می توانید مثلا از محتوای این نود استفاده کنید و این محتوا را به یک بردار embedding یا فیچر تبدیل کنید.

به تعدادی لایه هایی که این content را به یک بردار ویژگی سطح صفر برای GNN تبدیل می کنند زیاد کنید به این کار preprocessing بود

همینطور می توانید بعد از این که تعدادی لایه GNN استفاده کردید، خروجی لایه GNN را از یک سری شبکه عصبی معمولی عبور بدهید ← post_processing

pre_processing معمولا خیلی مهم، خیلی مفید و قابل اجراست وقتی که نودهای شما یک content خیلی غنی دارند. مثلا هر نود text و عکس و ... دارد.

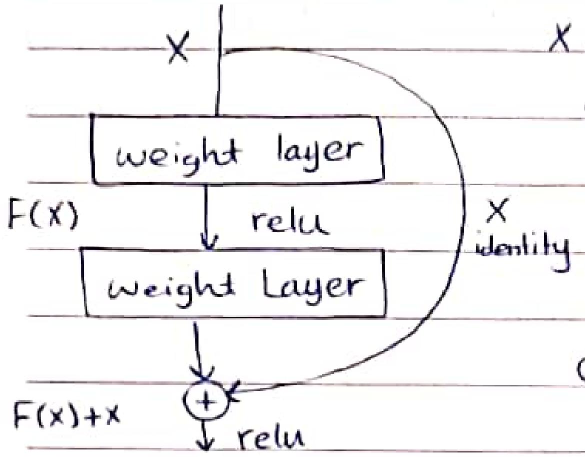
post_processing معمولا وقتی مفیده که تسک down stream شما مشخص هست مثلا می خواهید prediction انجام بدهید. تبدیل بردار embedding به prediction را می توانید با استفاده از یک Single Layer انجام بدهید یا می توانید چندتا لایه انجام بدهید که حتی شبکه عصبی پیچیده ای داشته باشید دوباره.



معمولا خیلی مفیدند این لایه های non_GNN و خیلی کارایی ارزشمندی دارند.

یه راه دیگه: Skip Connections (اتصالات پرشی)

در حالت استاندارد خروجی یک لایه را به ورودی لایه‌ی بعدی وصل می‌کنید.



در لایه‌ی l-1 یک embedding برای نود v به نام X تولید کرده اید. این تعبیه را به لایه‌ی l می‌دهید و یک embedding در لایه‌ی l برای X تولید می‌کنید.
در حالت استاندارد شما X را به F می‌دهید F(x) تولید می‌شود و ادامه می‌دهید
ولی در حالت skip Connection هر دو حالت را ادغام می‌کنید.

X را می‌دهید به لایه‌ی جدید و F(X) تولید می‌شود و همینطور X را از روی لایه‌ی l (لایه‌ی جدیدتان) عبور می‌دهید. در نهایت مثلا این‌ها را با هم جمع می‌کنید یا از یک AGG دیگری استفاده می‌کنید.
یعنی میشه خروجی لایه‌ی l-1 را با خروجی لایه‌ی l جمع (یا AGG) کنید
خروجی شما بعد از لایه‌ی l این فرم را می‌گیرد. F(X) + X

۲ دلیل وجود دارد که این skip Connection ها در عمل مفید واقع می‌شوند:

① ممکنه لایه‌های ابتدایی ما لایه‌های واقعاً خوبی باشند. این فرآیند به ما کمک می‌کنه از embedding ها لایه‌های پایین که هنوز پدیده over-smoothing برایشان رخ نداده استفاده کنیم

② به طور کلی شبکه‌های علت خوب کار کردنشان این است که فضای Search بزرگی را تولید می‌کنند و در این فضا دنبال مقادیری برای پارامترها می‌گردند که تابع هدف را بهینه کنند. هر چقدر این فضای جست و جو بزرگتر باشد از نظر accuracy بهتر است و کارایی بالاتره.
وقتی skip Connection انجام می‌دهید فضای حالت بزرگتری دارید. تعداد حالات ممکن بیشتره.
مثلا برای N=3 حالت های skip connection در اسلاید رسم شده
در حقیقت شما skip connection هم حالت‌های shallow را در نظر می‌گیرد و هم حالت‌های deep را. یعنی به صورت همزمان تست می‌کنید که تعداد لایه‌ها چند تا باشه بلکه به مدل این اجازه را می‌دهید که [illegible] N لایه را در نظر بگیرد و در هر کدام این حالات بهینه را انتخاب کند.

skip Connection به شیوه‌های مختلفی میتونه انجام بشه
مانند حالت‌های مختلف مثل non Linearity را یا بعد یا قبل Connection اعمال کنید

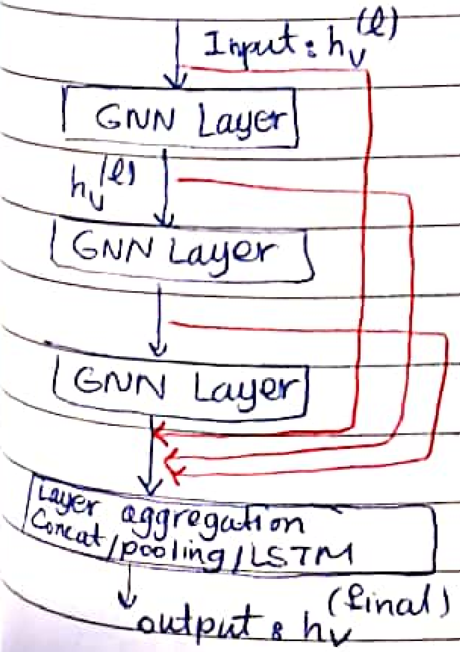
standard GCN : $h_v^{(l)} = \sigma\left(\sum_{u \in N(v)} W^{(l)} \frac{h_u^{(l-1)}}{|N(v)|}\right)$

به هر کدام از الگوریتم‌های موجود میتوانید skip Connection را اضافه کنید
اگر در GCN استفاده کنیم:

$$h_v^{(l)} = \sigma\left(\underbrace{\sum_{u \in N(v)} W^{(l)} \frac{h_u^{(l-1)}}{|N(v)|}}_{F(X)} + \underbrace{h_v^{(l-1)}}_{X}\right)$$

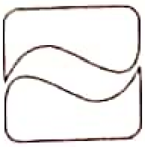
آیا باید embedding لایه قبل بدون اینکه از non Linearity عبور کند جمع شود؟
هر دو حالت در عمل می‌تونه استفاده بشه. در عمل میتونه از روی بعضی ماژول‌های آن لایه عبور کنید یا از روی بعضی‌ها skip کنید

skip Connection را به گونه‌ی دیگری هم مطرح کرده‌اند:
پرش به جای اینکه بعد از لایه‌ی پیش رو باشه، به لایه‌ی آخر باشه
عملکرد در بعضی کاربردها مثلی کمتر جواب بهتر بده ولی بعضی این حالت
یعنی ما این حالت در هر لایه ۲ حالت دارید: یا وارد لایه‌ی بعدی پیش روی می‌روید یا اینکه از روی کل لایه‌های پیش روی می‌پرید و بعد از خروجی لایه‌ی آخر می‌روید.
بعد از لایه‌ی آخر یه سری aggregatorهای مختلف دارید



تو آخر یه سری embedding دارید که تعدادشان به تعداد کل لایه‌هاست. در نهایت باید این‌ها را با هم aggregate کنید.

مثلاً شکل روبه‌رو k تا embedding وارد box آخر می‌شود
و یک embedding تولید می‌شود



نکته بعدی در Design space شبکه های عصبی گرافی:

گراف ما (یا همان دیتاست ورودی (Training) ما) به صورت یک دیتای آموزشی مناسب و ایده آل برای ساختن Computational Graph ها و یا embedding است.

معمولا کم پیش می آید که روی گراف ورودی عملیاتی انجام ندهیم و بتوانیم از روی آن خیلی خوب و با کیفیت embedding ها را بسازیم.

در نتیجه یک مرحله ای که در Design Space شبکه های عصبی گرافی داریم اصطلاحا Graph Manipulation (دستکاری گرافی) نامیده می شود.

Graph Manipulation از دو عمل اصلی تشکیل شده:

① Graph Feature Augmentation افزونه سازی ویژگی
② 〃 structure 〃 ساختاری 〃

تا الان فرض می کردیم که گراف ورودی ما برای ساخت تعبیه ها مناسبه. این فرض مشکلاتی دارد از جمله:

① نودهای ما ممکنه Feature Vector اولیه نداشته باشند. وقتی از GNN ها استفاده می کنیم فرض می کنیم که در لایه ی صفر یک بردار برای هر نود فراهم است. یک بردار فیچر یا embedding برای هر نود فراهم است.
ولی خیلی اوقات این بردار در لایه ی صفرم فراهم نیست. ما تنها چیزی که داریم لیستی از یال هاست که برای هر گراف داریم.
پس در این جا افزونه سازی ویژگی باید انجام بدهیم.

② ساختار گراف ما برای تولید GNN ها مناسب نباشه. ممکنه گراف ما خیلی اسپارس (تنک) باشد. در این حالت مشکلی که پیش میاد اینه که AGG خیلی معنی پیدا نمی کند. در این صورت جلسه ی مشورتی هر نود خیلی کوچک می شود و نظراتی که برای نود مرکزی داریم خیلی کم می شود، مشورتی دریافت نمی کند و نمی توانیم برای یک نود جلسه مشورتی خوبی را بسازیم. در حقیقت هم تعبیه با کیفیتی تولید نمی کنه.

Scanned with CamScanner

حالت دوم

وقتی گراف ما خیلی dense هست همسایگی یک نود خیلی بزرگ می شود و خیلی از عملیات که ما داریم به شدت کند می شوند. مثلا AGG خیلی کند می شود. اجرای GNN را با مشکل روبه رو می کند و از نقطه نظر محاسباتی به صرفه نیست.

حالت سوم:

وقتی گراف خیلی بزرگه و مقدار نودها خیلی زیاده و اصلا resource ها ما اجازه نمی دهند که Computation Graph را برای همه ی نودها بسازیم هم از نظر زمان اجرا و هم حافظه با مشکل روبه رو می شویم.

پس به طور کلی خیلی وقت ها اتفاق می افتد که لازمه گراف ورودی دستکاری بشه تا بتوان از روی این گراف حاصل شبکه عصبی گراف را تشکیل بدهیم.

جلسه بیست و ششم - ۱۴۰۰/۹/۳۰

وقتی هیچ اطلاعاتی در مورد ویژگی نودها نمی‌توانیم داشته باشیم از Feature augmentation بهره می‌گیریم

structure augmentation: وقتی گراف خیلی sparse است یک سری نودها یا یال‌های virtual اضافه می‌کنیم

وقتی گراف dense است به جای این که کل همسایگی را برای یک نود در نظر بگیریم، سمپل می‌کنیم

وقتی گراف خیلی بزرگ است به جای اینکه برای همه‌ی نودها computation Graph تشکیل بدهیم تعدادی نود را سمپل می‌کنیم و فقط برای آن‌ها می‌سازیم

وقتی هیچ اطلاعات اضافی به جز ماتریس یا لیست یال‌ها نداریم از [illegible] فیچر استفاده می‌کنیم راه حل‌ها:

① به هر نود مقدار یکسانی اختصاص بدهیم. مثلاً 1 یا صفر

② به هر نود از یک مجموعه‌ای (مثلاً اعداد 1 تا 5) یک لیبل اختصاص بدهیم (لیبل یا color)

③ به هر نود یک ID یکسانی اختصاص بدهیم. مثلاً اعداد 1 تا n را به نودهای 1 تا n اختصاص دهیم

می‌توانید برای هر نود از یک بردار یک بعدی استفاده کنید که فقط آیدی را در بر دارد (و در لایه‌های بعدی این بردار 1 بعدی map می‌شود به d بعدی)

یا اینکه در همان اول کار بردار فیچر سطح صفرشان را تعداد ابعادش را زیاد کنید با استفاده از one-hot vector

one hot vector ← برداری که یکی از خانه‌هایش 1 است و بقیه صفرند

اگر n تا ID داریم می‌توانید برای هر نود یک بردار به طول n تشکیل دهید و خانه‌ی مربوط به آیدی آن نود را 1 بگذاریم و بقیه‌ی خانه‌ها را صفر.

مقایسه

① Expressive power ← یعنی توانایی شبکه عصبی در اختصاص تعبیه‌های متفاوت به نودهایی که از نظر ساختار گراف متفاوتند.

راه حل ① از این نقطه نظر متوسط و معمولی است ولی راه حل ② قوی است. در راه حل ① به همه‌ی نودها آیدی یکسان می‌دهیم پس احتمال اینکه لزوماً تعبیه یکسان داشته باشند بیشتر می‌شود صرف نظر از اینکه از نظر ساختاری یکسانند یا متفاوتند. ولی در مدل one-hot، expressive power بالاست چون همان اول کار اصلاً اجازه نمی‌دهیم هیچ دو تا نودی تعبیه سطح صفر یکسانی پیدا کنند

② Inductivity ← یعنی قابلیت تعمیم به نودهای جدید (فرآیندی که learn کردید قابل تعمیم به نودهای جدید باشد)

استفاده از فیچرهای ثابت خاصیت Inductivity بالایی دارد. چون نود جدید که می‌سازد خیلی راحت یک بردار با اندازه‌ی 1 می‌سازید مقدارش هم مشخص (1 یا 0). به پارامترهای وزن که یاد گرفته شده‌اند می‌توانید برای این نود در نظر بگیرید

ولی one-hot پایین است، یک دلیل این است که طول بردار one-hot برابر تعداد نودهاست و دوباره باید از اول طول بردار را برای همه‌ی نودها تغییر بدهیم. حتی اگر از بردار one-hot هم استفاده نکنید باز هم چالش برانگیز است چون وزن‌هایی که برای نودهای دیگه یاد گرفته شده خیلی optimize شده باشن بر اساس ID اولیه.

③

Computational Cost ← روش constant خیلی کمینه است هم از نظر space و هم از نظر time ولی وقتی از one-hot استفاده می‌کنیم یک بردار اولیه بزرگ برای هر نود در نظر می‌گیریم هم از نظر space و هم time هزینه‌بر است. از نظر space ← بردار ویژگی اولیه بزرگه. از نظر time ← چون تعداد پارامترا که باید تعریف بشه در بردار اولیه رو map میکنه به بردار با ابعاد بالاتر، باید dxn باشه. تعداد پارامترهایی که باید train بشن خیلی زیاد می‌شه

④ Use cases (موارد استفاده) constant ← خیلی راحت در هر گرافی قابل استفاده است خصوصاً گراف‌های بزرگ

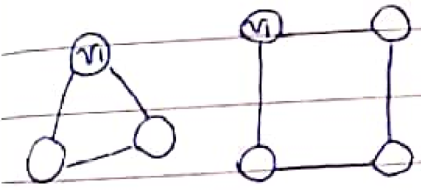
one-hot ← معمولاً برای گراف‌های کوچک استفاده می‌شود، setteing هم transductive است دیگه اینکه مقیاس‌پذیر نیست

چرا Feature Augmentation مهم است؟

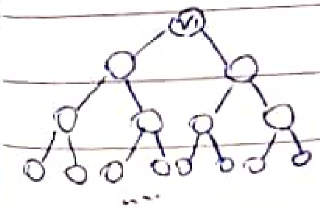
① شکل‌های خاصی گراف این طور ساخته شده‌اند که در لایه‌ی صفرشان یک برداری را دریافت کنند

② بعضی از ویژگی‌ها مثل بعضی ویژگی‌های نودها نمی‌توانند یاد گرفته شوند

مثال:



embedding نود v1 در گراف سمت چپ و سمت راست اگر feature vector های لایه‌ی صفر را یکی بگیریم در هر دو تا آیا تفاوت می‌کند؟ تفاوتی نخواهند کرد. در هر دو حالت Computation Graph به این صورت است:



این مورد بد است چون دو تا گراف متفاوت هستند باید embedding های متفاوتی داشته باشند. نودهای این دو تا گراف هم متفاوتند باید embedding های متفاوتی داشته باشن.

دو تا گراف وقتی خوبه که embedding های یکسانی داشته باشن که با هم isomorphic باشند

حتی نودهای این دو تا گراف هم خوبه که تعبیه‌های متفاوت داشته باشن چون یکی در یک سیکل با طول ۳ و یکی در یک سیکل با طول ۴ قرار دارد.

پس مقادیر اولیه نودها اگر یکسان در نظر بگیریم برای همه نودها، بعضی جاها expressiveness پایین می‌آید می‌توانید از یک بردار onehot استفاده کنید که مثلاً با 10 اندازه سیکل گراف [illegible]:

[0,0,0,1,0,0]
[0,0,0,0,1,0]

در این حالت embedding های متفاوتی پیدا می‌کنند

در عمل چه اتفاقی می‌افتد؟
برای گراف‌های بزرگ روش constant استفاده می‌شود ولی یک راه بهتر استفاده از یک سری معیارهای ساختار گرافی است.

برای هر نود یک بردار داریم که ویژگی های ساختاری آن نود را می دهد. مثلا خانه اول ضریب خوشه بندی آن نود را می دهد خانه دوم page rank، خانه سوم Betweenness centrality و ... را نشان بدهد.
خیلی خوب جواب می دهد برای مسئله های مثل classification

افزونه سازی ساختاری (structure augmentation)

مشکل اتصالی که وجود دارد sparse بودن گراف است در این حالت شعاع همسایگی ای که تعریف می سازد خیلی کوچک است یعنی نمی تواند به اندازه کافی اطلاعات جمع کند در نتیجه تعبیه های خیلی بی کیفیتی می شود.

در این حالت راه اضافه کردن یک سری virtual Edge می باشد. یک سری یال های مجازی بین نودها اضافه می کنند. یا برای هر نود خصوصا آن هایی که درجه پایینی دارند یک سری virtual edge در نظر می گیرند و به نودهای دیگر وصل می کنند در نتیجه اندازه شعاع همسایگی یک مقدار بزرگ تر می شود و message های بیشتری نود مرکزی دریافت می کند.

مثلا یک کاری که در گراف های Bipartite انجام می شود: بین هر دوتا نودی که یک مسیر به طول ۲ وجود دارد یک یال ایجاد می کنند.

گراف دوبخشی (Bipartite) ← گراف هایی هستند که مجموعه نودهای آنها به دو بخش متمایز V_1 و V_2 تقسیم می شوند به طوری که یک نود در V_1 فقط می تواند به یک یا چند نود در V_2 یال داشته باشد. نودهای V_1 با هم یال ندارند، نودهای داخل V_2 هم با هم یال ندارند.

مثلا شبکه ی Auther_paper. در حالت استاندارد اگر بخواهید برایش GNN بسازید message ای که برای هر نود دریافت می کنید از نودهای از نوع paper است. شما می خواهید از نویسنده های دیگر message دریافت کنید. از نودهای هم نوع می خواهید پیام دریافت کنید. پس بین هر دوتا نودی که در authors هستند و یک مسیر به طول ۲ بین شان وجود دارد یک یال virtual قرار می دهید. مسیر به طول ۲ یعنی حداقل یک paper هست که این دو نویسنده باهم نوشته اند. در این حالت ماتریس مجاورت اینگونه می شود: $A + A^2$

یک کار دیگری که انجام می گیرد این است که یک نود virtual ایجاد کنید و از آن نود به همه ی نودها یال اضافه می کنید. در نتیجه فاصله ی بین نودها خیلی کم شده، بین هر دو نود حداکثر فاصله می شود ۲. برای وقتی خوب است که نودها خیلی از هم دورند

مثلا random walk را اجرا می کنند که همسایگی نود را بسازند. اگر گراف خیلی اسپارس باشد همسایگی قابل توجهی برای یک نود نخواهید ساخت. برای یک نود با استفاده از random walk. بعد به نود virtual اضافه می کنید و به همه یا تعدادی از نودها وصل می کنید.

در عمل معمولا اضافه کردن virtual node خیلی خوب جواب می دهد

Neighborhud Sampling

تا این جا فرضی که داشتیم این گونه بود که یک فرمول A دارید. کل همسایه‌های نود را همراه خود نود در نظر می‌گیرید.

ولی اگر گراف خیلی dense باشد همسایگی ساخته شده برای بعضی از نودها خیلی بزرگ می‌شود. مثلاً اگر گراف کامل باشد برای هر نود اندازه همسایگی می‌شود O(|V|) ← هزینه محاسباتی بالایی دارد. در این حالت مخصوصاً برای نودهایی که همسایگی خیلی بزرگی دارند نباید کل همسایگی در نظر گرفته شود به جای آن طریقه مشورتی را انتخاب می‌کنند. مثلاً می‌توانید تعدادی به طور تصادفی انتخاب کنید.

دو تا امتیاز دارد Sampling:

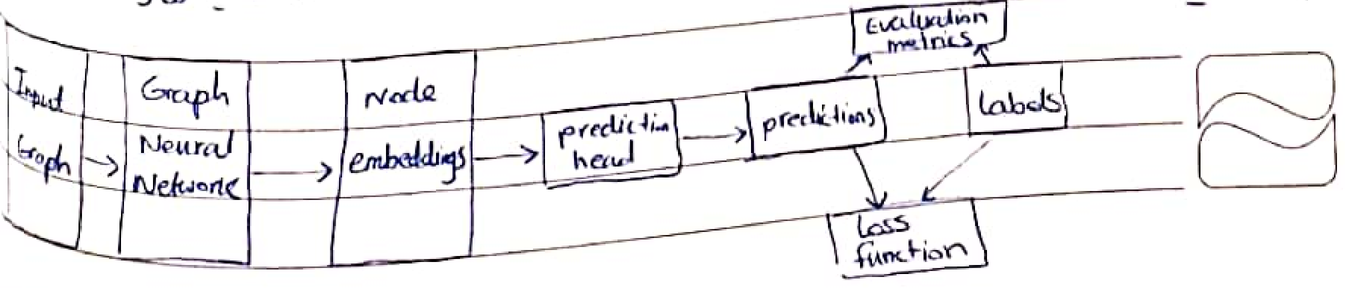
① به صورت expected value، تقریباً با Sampling به دست می‌آید همان تعبیه اصلی نود شماست یعنی شما حاصل می‌گیرید embedding شما یک متغیر تصادفی است پس یک امید ریاضی دارد که امید ریاضی این embedding برابر با embedding اصلی شماست.
مسئله این است که یک گراف وزن‌دار دارید، درجه وزن‌دار یک نود را حساب می‌کنید. مثلاً به نودی تعداد زیادی یال داریم و هر یال هم یک وزنی دارد. به جای اینکه وزن تک تک یال‌ها را جمع کنید، اگر O(|V|) تا یال باشد کار زیادی می‌شه. شما باید تعدادی یال به صورت تصادفی انتخاب می‌کنید وزن را حساب می‌کنید و حاصل را scale می‌کنید برای کل همسایه‌های این نود تخمین می‌زنید. این جا هم دقیقاً همان کار را انجام می‌دهید.

② روش‌های approximate خیلی هزینه‌ی محاسباتی پایینی دارند نسبت به روش‌های exact خصوصاً وقتی گراف شما خیلی بزرگ باشد. مثلاً یک صدم این حالت زمان نیاز دارد.

یکی از قسمت‌های اصلی که در Design Space شبکه‌های عصبی گرافی مطرح می‌شه اینه که چطور یک GNN را Train بکنیم:

GNN Training pipeLine

فرض کنید embedding ای که برای گراف تولید می‌کنیم برای تسک خاصی نباشد و General purpose باشد یعنی برای تعبیه با کیفیت خوب تولید می‌کنیم تعبیه‌ها تسک‌های downstream متفاوتی داریم مثل link prediction، clustering، classification، Regression، ... هرکدام از این‌ها می‌خواهند از این تعبیه استفاده کنند. یک گراف ورودی داریم، به یک GNN داده می‌شه و یک سری وزن یاد گرفته می‌شود از روی وزن‌ها یک سری embedding به دست میاد.



دیدیم که در لایه‌های بالا دیدیم که embedding با کیفیت خوب هستش که اصطلاحا similarity preserving هستن. یعنی این بردارها باید similarity preserving باشند.

از این به بعد باید این‌ها را به تسک‌های مختلف بدهیم. مثلا به ویژه تسک‌ها prediction هستند مثلا Link prediction، Node/Graph/Edge classification و Regression در سطح نود، گراف یا یال این بردارها به شبکه‌ها یا کامپوننت‌های دیگه‌ای که هم باشه بدهیم به عنوان کامپوننت‌های تسک downstream.

به prediction head طوری که در واقع همان لایه پیش‌بینی است که در شبکه‌های عصبی معمولی داریم. که بردار embedding های آخر را map می‌کند به سمت کلاس.

prediction head معمولا بردار embedding را به عنوان ورودی می‌گیرد و خروجی‌اش هم یک بردار prediction است. که توسط GNN تولید شده

یک loss function در GNN داشتیم که کمک می‌کرد embedding های خوبی با کیفیت تولید کنیم

یک loss function دوم هم در آخر داریم: نقش این loss function دوم برای یادگیری وزن‌های prediction head (لایه classification) است

۳ نوع prediction head داریم: ① در سطح نود ② در سطح یال ③ در سطح گراف

Node Level prediction

ورودی ← embedding یک نود   خروجی ← بردار prediction

Edge Level prediction

ورودی ← embedding دو endpoint یال   خروجی ← بردار prediction برای یال

Graph Level

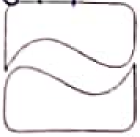
ورودی ← embedding تک تک نودهای گراف   خروجی ← بردار prediction

node-level به سرراست‌ترین و راحت‌ترین حالت است. به تابع head شبیه کارهای لایه‌های classification یا Regression را انجام می‌دهد. ورودی‌اش آخرین نمایش تولید شده توسط GNN برای نود v است و خروجی‌اش بردار prediction است.

$$\hat{y}_v = \text{Head}_{node}(h_v^{(L)}) = W^{(H)} h_v^{(L)}$$

$\hat{y}_v$ ← مثلا میتونه یه فضای k بعدی باشه

$h_v^{(L)}$ ← نمایش نهایی embedding است مثلا d بعدی



تابع Head عملا بردار d بعدی فضای embedding را به بردار k بعدی فضای prediction می‌برد

اگر تسک رگرسیون باشه k=1 است یک عدد را پیش‌بینی می‌کنه

Head می‌تواند انواع مختلفی از توابع را داشته باشد مثلا پرسپترون یا GNN و ...

حالت ساده‌اش: $W^{(H)} h_v^{(L)}$. یه ماتریس W داریم که trainable است و یک پرسپترون را شبیه‌سازی می‌کند. ورودی‌اش بردار در فضای d بعدی و خروجی‌اش بردار در فضای k بعدی است: $W_{k \times d}$

یک loss function تعریف می‌کنند و با optimize کردنش W را یاد می‌گیرند.

این دقیقا نقش همان classification layer را بازی می‌کند

Edge-Level ← خروجی یه بردار k-way prediction

ورودی‌های تابع embedding نود u و نود v $\quad \hat{y}_{uv} = \text{Head}_{edge}(h_u^{(L)}, h_v^{(L)})$

در این حالت تابع Head چگونه باید باشد؟

① embedding نود u و نود v را concat کنیم، یه بردار با 2d خانه خواهیم داشت.
بعد از یک single layer و یک ماتریس وزن استفاده می‌کنیم که بردار در فضای 2d را به فضای k بعدی بهمون می‌دهد. ابعاد ماتریس: $k \times 2d$

(Concatenation + Linear)

② Dot product

گزینه دوم: ② dot product و یا ماتریس‌های trainable می‌توانند داشته باشند

مثلاً در مسئله downstream تان دیتای برچسب دار ندارید در نتیجه فاز training نمی‌توانند داشته باشند. در این حالت می‌توانید شباهت دو تا بردار embedding را در نظر بگیرید و خروجی بدهید. مثلاً با dot product

دو تا ایراد دارد این روش:

① کار درستی نیست در حالتی که دیتای برچسب دار ندارید. اگر داده برچسب دار داشته باشید خوب است که از آن استفاده بکنید. فاز training داشته باشید و prediction را نسبت به دیتایی که دارید optimize بکنید.

② خروجی ما همیشه one_way است در حالی که ممکنه بخواهیم k way prediction داشته باشیم. مقدار خروجی بردار prediction ممکنه بخواهیم که بیشتر از یک بعد باشد. یک روش پیشنهاد شده:

K تا ماتریس وزن تعریف می‌کنند: $W^{(1)}, \dots, W^{(k)}$

توسط $W^{(1)}$ خانه اول و ... توسط $W^{(k)}$ خانه k ام به وجود می‌آید

$$\hat{y}_{uv}^{(1)} = (h_u^{(L)})^T W^{(1)} h_v^{(L)}$$

ابعاد $d \times d \leftarrow W^{(1)}$

$\dots$

$1 \times d \leftarrow (h_u^{(L)})^T$

$$\hat{y}_{uv}^{(k)} = (h_u^{(L)})^T W^{(k)} h_v^{(L)}$$

ابعاد $d \times 1 \leftarrow h_v^{(L)}$

$$\hat{y}_{uv} = \mathrm{Concat}(\hat{y}_{uv}^{(1)}, \dots, \hat{y}_{uv}^{(k)}) \in \mathbb{R}^k$$

در نهایت یک بردار در فضای k بعدی دارید

در اینجا ماتریس‌های trainable دارید $W^{(1)}$ تا $W^{(k)}$

Graph_Level prediction

[illegible]

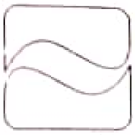
یک گراف داریم که برای تک تک نودهایش embedding حساب شده ← ورودی تابع Head

خروجی ← یک بردار در فضای k بعدی

$$\hat{y}_G = \mathrm{Head}_{graph}(\{h_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$$

در این‌جا تابع head شبیه aggregation است که یک ماتریسی از بردارهای embedding می‌گیره به عنوان ورودی و آن را map می‌کند به یک تک بردار. این جا هم دقیقاً همان است. به جای Head از همه توابعی که برای Aggregation می‌توانستیم استفاده کنیم استفاده می‌کردیم

مثال: mean, max, sum, MLP, LSTM



که از این توابع بهره می‌گیریم تا یک تک بردار در فضای k بعدی بدست بیاوریم یعنی با یک تبدیل خطی این بردارها را به فضای k بعدی می‌بریم. این روش‌ها برای گراف‌های کوچک خوب کار می‌کنند.

مشکلی که این روش ها برای گراف های خیلی بزرگ دارند این است که اطلاعات embedding ها از بین برود.

یعنی ممکنه دو تا گراف که نودهای کاملا متفاوتی دارند بعد از اعمال این عملیات embedding هاشون که پیدا می کنند به هم خیلی شبیه باشه یا عین هم باشند.

راه حل:

به صورت hierarchical (سلسله مراتبی) اعمال کنید و این گام ها را دنبال کنید:

۱) برای تک تک نودهای گراف یک embedding تولید کنید

۲) یک partitioning روی نودهای گراف انجام بدهید. (با یک معیاری) نودهای شبیه هم را باید partition قرار بدهید

۳) از روی embedding نودهای داخل یک پارتیشن یک embedding برای خود آن پارتیشن ایجاد کنید. مثلا می توانید sum بگیرید و از حاصل ReLU بگیرید

مثال: $\{\underbrace{-1,-2}_{a},\underbrace{0,1,2}_{b}\}$

$\hat{y}_a = ReLU(Sum(\{-1,-2\}))=0$

$\hat{y}_b = ReLU(Sum(\{0,1,2\}))=3$

$$\hat{y}_G = ReLU(Sum(\{y_a,y_b\}))=3$$

مثال: $\{\underbrace{-10,-20}_{a},\underbrace{0,10,20}_{b}\}$

$\hat{y}_a = ReLU(Sum(\{-10,-20\}))=0$

$\hat{y}_b = 30$

$\hat{y}_G = 30$

همین جا متوقف می شویم چون یک تک نود داریم در لایه ی آخر

برای پارتیشن بندی یک پیشنهاد که شده اینه که از یک شبکه عصبی گرافی دیگر استفاده کنید و از embedding های محتمل برای نودها تولید کرده باید استفاده کنید.
یک GNN دیگه که تابع هدفش این نیست که embedding های General تولید کنه بلکه باید embedding هایی که برای clustering مناسب هستند را تولید کنند.

بعد از تولید پارتیشن ها از روی embedding های هر پارتیشن یک embedding می سازید. در سطح بعد این فرآیند را تکرار می کنید تا آن جایی که نهایت به یک پارتیشن و یک embedding برسیم